HANDBOOK

Web Developer

Grade XI – IT-ITeS Vocational Education

Web Essentials • HTML • CSS • JavaScript • SDLC

A structured textbook guide designed for practical web development mastery.

Based on Grade XI Web Developer study material topics

Contents

1 Web Design and Development **1**
1.1 Understanding Web Design . . . 1
1.1.1 User Interface and User Experience . . . 1
1.2 Understanding Web Development . . . 2
1.2.1 Static and Dynamic Websites . . . 2
1.3 The Web Design and Development Life Cycle . . . 3
1.3.1 Stage 1: Discover the Need . . . 3
1.3.2 Stage 2: Plan the Work . . . 3
1.3.3 Stage 3: Design the Structure and Appearance . . . 3
1.3.4 Stage 4: Develop the Website . . . 3
1.3.5 Stage 5: Test and Review . . . 4
1.3.6 Stage 6: Publish and Deploy . . . 4
1.3.7 Stage 7: Maintain and Improve . . . 4
1.4 Core Principles of Effective Web Design . . . 4
1.4.1 Visual Hierarchy . . . 4
1.4.2 Consistency and Simplicity . . . 4
1.4.3 Colour and Typography . . . 4
1.4.4 Navigation and Content . . . 4
1.5 Responsive Design and Device Support . . . 4
1.6 Accessibility and Inclusive Design . . . 5
1.7 Designing for Quality, Security and Performance . . . 5
1.8 Example: Planning a Learning Centre Website . . . 5
1.9 Chapter Review . . . 6

2 HTML: Document Architecture **7**

3 CSS Layouts and Box Model **8**

4 JavaScript Fundamentals **9**

5 Requirements, Planning and Project Documents **10**
5.1 Understanding the Need . . . 10
5.1.1 Stakeholders and Users . . . 10
5.1.2 Functional and Non-functional Requirements . . . 10
5.2 Brief Requirements Document . . . 10
5.3 Planning and Scope . . . 10
5.4 Business and Software Requirements . . . 11
5.5 High-Level and Low-Level Design . . . 11
5.6 Design Models and Interfaces . . . 11

6 Web Technologies, Browsers and HTTP **12**
6.1 How a Web Page Reaches a Browser . . . 12
6.2 HTTP and HTTPS . . . 12

6.2.1 Common Status Codes . . . 12
6.3 Browser Developer Tools . . . 12
6.4 Development Tools and Standards . . . 12
6.5 Common Application Development Models . . . 13

7 Semantic HTML and Accessible Structure 14
7.1 Meaningful HTML Elements . . . 14
7.2 Headings and Content Hierarchy . . . 14
7.3 Lists and Grouped Information . . . 14
7.4 Links, Images and Alternative Text . . . 15
7.5 Forms and Labels . . . 15

8 CSS Selectors, Cascade and Responsive Layout 16
8.1 Selectors . . . 16
8.2 Cascade, Specificity and Inheritance . . . 16
8.3 Responsive Design . . . 16
8.4 Flexbox and Grid . . . 17

9 JavaScript Variables, Types and Operators 18
9.1 Storing Values . . . 18
9.2 Common Data Types . . . 18
9.3 Operators and Comparisons . . . 18
9.4 Readable and Safe Code . . . 18

10 Conditions, Loops, Functions and Arrays 19
10.1 Conditional Decisions . . . 19
10.2 Loops . . . 19
10.3 Functions . . . 19
10.4 Arrays and Objects . . . 19
10.5 Working with Strings . . . 19

11 DOM, Events and Form Interaction 21
11.1 The Document Object Model . . . 21
11.2 Events . . . 21
11.3 Validation and Feedback . . . 21
11.4 Safe DOM Updates . . . 21
11.5 Drawing with the HTML Canvas . . . 21
11.6 Using JavaScript with Images . . . 22

12 Testing, Debugging and Quality Assurance 23
12.1 Why Testing Matters . . . 23
12.1.1 Useful Test Categories . . . 23
12.2 A Practical Debugging Method . . . 23
12.3 Test Case Example . . . 24

13 Accessibility, Performance and Secure Publishing 25
13.1 Accessibility Essentials . . . 25
13.2 Performance Basics . . . 25
13.3 Security Awareness . . . 25
13.4 Publishing Checklist . . . 25

14 Integrated Mini Project: Learning Centre Website 26
14.1 Project Brief . . . 26

14.2 Suggested Page Structure 26
14.3 Implementation Plan 26
14.4 Evaluation Rubric 26

Chapter 1

Web Design and Development

A website often begins with a simple question: what should people be able to find or do here? The answer shapes every decision that follows, from the first sketch of a page to the code that runs in a browser. Good web design helps visitors understand where they are and what they can do next. Good development turns that plan into a working site, checks that it behaves properly, and makes it dependable on different devices.

The two activities work best together. A page may look attractive but still frustrate people if its navigation is confusing or its text is difficult to read. Likewise, a technically correct page is not very useful if visitors cannot find the information they came for. A thoughtful developer keeps the people using the website in mind while planning, designing, coding, testing, and improving it.



Figure 1.1. A web developer's workspace. Photograph: Shixart1985, Wikimedia Commons, CC BY 2.0.

1.1 Understanding Web Design

Web design concerns the organisation and presentation of information on a website. It includes page layout, colour, typography, images, navigation, spacing, and the way a visitor moves from one task to another. Good design makes the purpose of each page clear and helps users find what they need without unnecessary effort.

A designer begins by asking who will use the website, what those people need to do, and what information they expect to find. A school website, for example, may need clear links to admissions, courses, notices, and contact details. An online shop needs product information, prices, search and filtering, a cart, and a clear checkout path. The design should reflect the real purpose of the site rather than decorative trends alone.

1.1.1 User Interface and User Experience

The user interface (UI) is the part of the system with which a person interacts: menus, buttons, links, forms, headings, and visual controls. User experience (UX) is the broader experience of using the site, including how easy it is to learn, how quickly a task can be completed, and whether the process feels predictable and trustworthy.

A visually attractive interface can still provide a poor experience if its navigation is confusing or its forms give unclear errors. Similarly, a simple interface can provide a strong experience when the content is well organised, controls are labelled clearly, and users receive helpful feedback after an action.

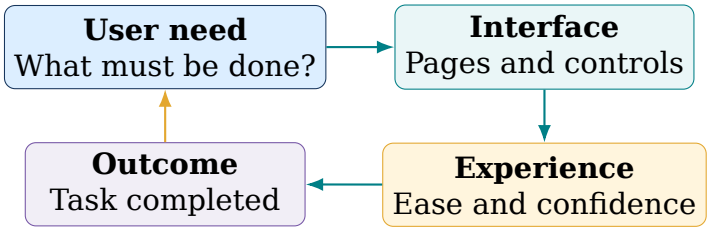


Figure 1.1. UI elements support a user experience that should lead to a clear outcome.

1.2 Understanding Web Development

Web development is the process of building and maintaining the website's structure, appearance, behaviour, and supporting services. Front-end development creates the part that runs in the browser. HTML describes the content and its meaning, CSS controls presentation and layout, and JavaScript adds behaviour such as menu interaction, input validation, and dynamic updates.

Back-end development, when required, handles server-side tasks such as business rules, user authentication, data storage, and communication with other services. A database stores information in an organised form. An application programming interface (API) can provide a defined way for the browser or another application to request data or perform an operation. A small static information site may not need a back end; a learning portal with student accounts and saved results usually does.

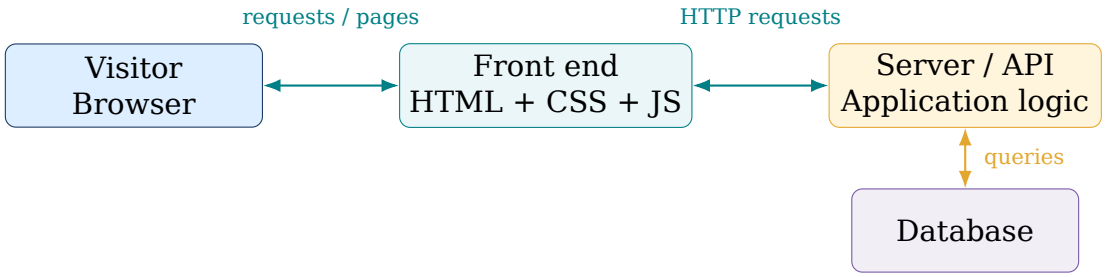


Figure 1.2. A common website architecture. A simple static site may use only the browser and front-end files.

1.2.1 Static and Dynamic Websites

A static website serves pages whose content is mainly prepared in files such as HTML, CSS, JavaScript, and images. It is suitable for portfolios, simple information pages, event announcements, and small documentation sites. A dynamic website can change its output based on user input, stored data, permissions, or other conditions. Examples include online banking, booking systems, shopping sites, and student portals.

The distinction is about how the content is produced, not whether the page has styling or animation. A static page can be visually rich and interactive in the browser. A dynamic service often combines browser code with server-side processing and a database.

1.3 The Web Design and Development Life Cycle

The development life cycle provides a sequence of activities that helps a team move from an idea to a reliable website. The stages may overlap, and teams often return to earlier decisions after testing or feedback. The purpose of a life cycle is not to create paperwork for its own sake; it is to reduce avoidable mistakes, make decisions visible, and ensure that the final site meets its requirements.

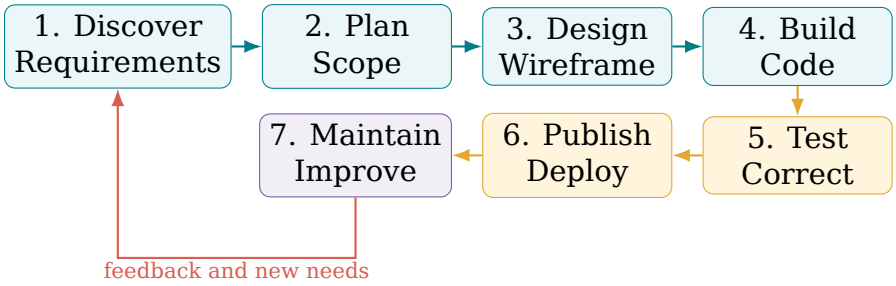


Figure 1.3. A practical website life cycle with feedback after release.

1.3.1 Stage 1: Discover the Need

Identify the problem the website should solve and the people who will use it. Gather information from stakeholders, review existing material, and list the most important user tasks. For a school information site, these tasks may include finding a course, checking an announcement, or contacting the office. Clear goals prevent the project from becoming a collection of unrelated pages.

1.3.2 Stage 2: Plan the Work

Define the scope, deliverables, schedule, resources, and responsibilities. Decide which features are essential for the first version and which can wait. A page list and a simple task board can be sufficient for a small project. Planning should also consider risks such as missing content, unavailable images, device compatibility, privacy, and limited time.

1.3.3 Stage 3: Design the Structure and Appearance

Create a sitemap to show how pages relate to one another. Sketch wireframes to decide where headings, navigation, content, forms, and images should appear. Choose a consistent colour palette, type scale, spacing system, and button style. Review the design against the user tasks before investing time in detailed implementation.

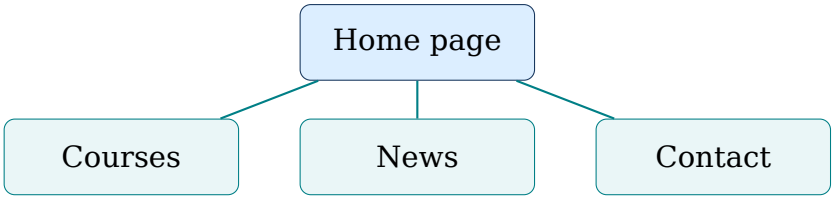


Figure 1.4. Example sitemap for a small educational website.

1.3.4 Stage 4: Develop the Website

Build the page structure with semantic HTML, apply CSS for layout and responsive behaviour, and add JavaScript where interaction is needed. Keep files organised and use meaningful names. Work in small steps: create a page shell, check it in a browser, add one feature, and test again. This makes it easier to identify which change introduced a problem.

1.3.5 Stage 5: Test and Review

Testing checks whether the site behaves as intended. Confirm that links work, forms handle missing or invalid values, images load, text is readable, and the layout works on narrow and wide screens. Use keyboard navigation and inspect error messages in the browser's developer tools. Fix serious problems before publishing, then repeat the relevant tests after changes.

1.3.6 Stage 6: Publish and Deploy

Publishing makes the site available to its intended audience. Deployment may involve uploading static files to a hosting service, connecting a domain name, enabling HTTPS, configuring server settings, and checking that all paths work on the live site. A site that works on a developer's computer can still fail after deployment if file names, capitalisation, paths, permissions, or environment settings differ.

1.3.7 Stage 7: Maintain and Improve

After release, check for broken links, outdated information, security updates, performance problems, and user feedback. Keep a copy of the source files and a record of important changes. Maintenance is part of development, not an optional activity that begins only when something breaks.

1.4 Core Principles of Effective Web Design

1.4.1 Visual Hierarchy

Visual hierarchy helps readers recognise what is most important. Main headings should be more prominent than body text; related items should be grouped; and primary actions should be easy to identify. Use size, weight, spacing, alignment, and colour consistently. Avoid giving every element the same visual emphasis, because this makes it harder to scan a page.

1.4.2 Consistency and Simplicity

Use the same navigation pattern, heading styles, button shapes, and spacing rules across related pages. Consistency reduces the effort required to learn the site. Simplicity does not mean removing necessary information; it means presenting the information in a clear order and avoiding decoration that competes with the task.

1.4.3 Colour and Typography

Choose colours that support the site's purpose and provide sufficient contrast between text and its background. Do not communicate important status using colour alone; pair colour with a label or icon where appropriate. Use a small, consistent set of readable fonts and line lengths. Body text should remain comfortable to read on both desktop and mobile displays.

1.4.4 Navigation and Content

Navigation labels should describe their destination, for example, "Courses" or "Contact", rather than vague labels such as "Click here". Put frequently used destinations where people expect them. Write concise headings, divide long material into meaningful sections, and ensure that images have a clear purpose. Content should answer the questions visitors came to resolve.

1.5 Responsive Design and Device Support

A responsive website adapts its layout to the available screen space. A multi-column desktop layout may become a single column on a phone; navigation may wrap or

collapse; images should scale without distortion; and buttons should remain large enough to tap comfortably. Responsive design uses flexible layouts, relative sizing, and CSS media queries rather than building an entirely separate page for every device.

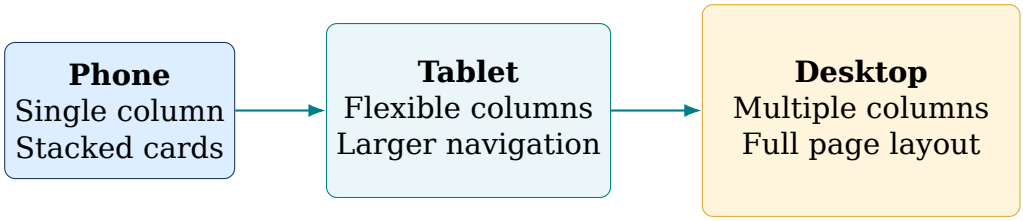


Figure 1.5. The same content can adapt to different viewport sizes.

1.6 Accessibility and Inclusive Design

Accessible design allows people with different abilities and ways of interacting to use a website. Provide meaningful headings, labels for form controls, alternative text for informative images, sufficient colour contrast, and visible keyboard focus. Ensure that interactive elements can be reached and operated without a mouse. Plain, direct language also helps users who are new to the subject or reading in a second language.

Accessibility is best considered during planning and design, then checked throughout development. Retrofitting it at the end may require changing page structure and interaction patterns. A practical review includes keyboard-only navigation, zooming text, checking labels, and verifying that the purpose of each link is understandable.

1.7 Designing for Quality, Security and Performance

Quality includes correctness, usability, reliability, maintainability, accessibility, and performance. Optimise images so that they are not unnecessarily large, avoid loading resources that are not needed, and test pages on a realistic connection. Keep dependencies and source files organised so another developer can understand the project.

Security must also influence design decisions. Collect only the information the site actually needs, explain why it is collected, and avoid placing passwords, private keys, or secret tokens in front-end code. Use HTTPS for deployed sites and apply appropriate validation and access controls when server-side features are involved. A polished interface cannot compensate for unsafe handling of user data.

1.8 Example: Planning a Learning Centre Website

Consider a learning centre that needs to publish courses, class schedules, announcements, and contact information. Before coding, the team identifies the main visitors: learners, parents, and staff. The essential pages are Home, Courses, Notices, and Contact. The home page introduces the centre and links to the most common tasks; the courses page provides names, descriptions, and eligibility information; the notices page lists dated updates; and the contact page gives verified contact details.

The first version can be a static website if the information is updated manually. If staff later need to sign in and manage notices, the project may require authentication, a server, and a database. This illustrates an important planning rule: begin with the simplest solution that satisfies the confirmed need, then add complexity only when a real requirement justifies it.

1.9 Chapter Review

1. Distinguish web design from web development and give two examples of work done in each area.
2. Explain the difference between user interface (UI) and user experience (UX).
3. Describe the role of HTML, CSS, JavaScript, a server, an API, and a database in a website.
4. Compare static and dynamic websites and provide one suitable example of each.
5. List the major stages of the web design and development life cycle.
6. Explain why requirements and scope should be discussed before coding begins.
7. What is a wireframe, and how does it help a development team?
8. Describe four principles of effective web design.
9. Explain responsive design and name two ways a page can adapt to a phone screen.
10. Identify four accessibility checks that should be completed before launch.
11. Why should testing and maintenance continue after a website is published?
12. For a learning centre website, identify the main users, essential pages, and one feature that could be postponed to a later release.

Chapter 2

HTML: Document Architecture

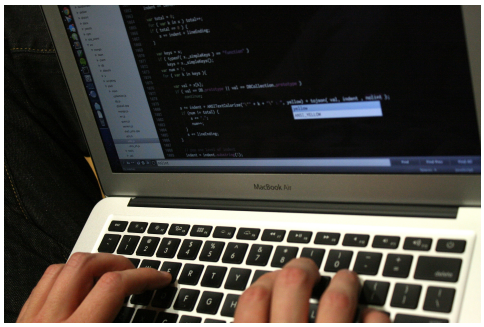


Figure 2.1. A learner working with HTML and CSS code. Photo: hackNY / Matylda, CC BY-SA 2.0. HyperText Markup Language (HTML) provides the fundamental skeletal structure for web content. Elements encapsulate textual data and media objects using semantic tags.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Developer Handbook</title>
</head>
<body>
  <header>
    <h1>Web Development Basics</h1>
  </header>
  <main>
    <p>Semantic document structures improve accessibility and indexing.</p>
  </main>
</body>
</html>
```

Review Questions

1. What purpose does the `<!DOCTYPE html>` declaration serve?
2. Write out a basic semantic HTML skeleton including a header, main container, and footer.

Chapter 3

CSS Layouts and Box Model



Figure 3.1. Responsive design adapts a website to desktop, tablet and phone screens. Photo: Muhammad Rafizeldi, CC BY-SA 3.0. Cascading Style Sheets (CSS) control visual presentation, spacing, typography, colors, and layout positioning across device viewports.

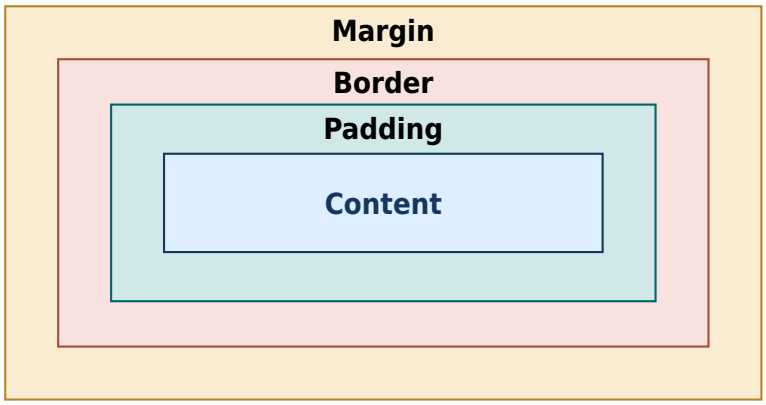


Figure 4.1. Visual layers of the CSS Box Model.

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
.card {
  width: 100%;
  max-width: 400px;
  padding: 1.5rem;
  border: 1px solid #007F86;
  background-color: #EAF6F7;
}
```

Review Questions

1. Describe the four components comprising the CSS Box Model.
2. Explain how `box-sizing: border-box` simplifies spatial calculations.

Chapter 4

JavaScript Fundamentals



Figure 4.1. Source code is written and reviewed in a programming editor. Photo: Martin Vorel, CC BY-SA 4.0. JavaScript enables dynamic interactive behaviors, state manipulation, event processing, and asynchronous network communication directly within client browser environments.



Figure 5.1. The triad of front-end web development technologies.

```
const calculateTotal = (price, taxRate) => {
  if (price <= 0) {
    throw new Error("Invalid price value provided.");
  }
  return price + (price * taxRate);
};

console.log(calculateTotal(100, 0.18));
```

Review Questions

1. Contrast `let` and `const` variable scoping rules.
2. Explain how event listeners bind user actions to script logic.

Chapter 5

Requirements, Planning and Project Documents

5.1 Understanding the Need

A web project begins by understanding the problem it is expected to solve. A website for a school, a small shop, a public service office, and an online learning platform may all use a browser, but each has different users, content, rules, and success measures. Clear requirements prevent avoidable rework and help a team agree on what must be delivered.

5.1.1 Stakeholders and Users

A stakeholder is a person or group affected by the project. Stakeholders may include the customer, website visitors, content editors, administrators, support staff, and the organisation responsible for maintaining the site. Users should be described by their goals rather than by assumptions about their age, device, or technical knowledge.

5.1.2 Functional and Non-functional Requirements

Functional requirements describe what the system must do: display a product catalogue, validate a form, search for an article, or allow an authorised editor to update content. Non-functional requirements describe qualities and constraints, such as response time, accessibility, security, browser support, reliability, and ease of maintenance.

Requirement type	Example for a school website
Functional	A visitor can view announcements and download the academic calendar.
Usability	Navigation labels are understandable to first-time visitors.
Accessibility	Content can be navigated using a keyboard and read by assistive technology.
Performance	Pages avoid unnecessarily large images and scripts.
Security	Administrative editing features require authentication.
Maintainability	Repeated page elements use consistent templates and styles.

5.2 Brief Requirements Document

A simple project brief should record the purpose, target audience, main pages, required functions, content owner, constraints, and acceptance criteria. Acceptance criteria are observable conditions used to decide whether a feature is complete. For example, “the contact form is finished” is vague; “the form identifies missing required fields and displays a clear confirmation after valid submission” is testable.

5.3 Planning and Scope

Scope defines what is included and what is excluded. A small first release may include a home page, about page, services page, and contact form. A payment system or user account feature may be planned for a later release. A scope list helps the team manage time and avoid adding unplanned work without reviewing its impact.

5.4 Business and Software Requirements

A Business Requirements Document (BRD) describes the business need, intended users, goals, scope, constraints, and expected outcomes. A Software Requirements Specification (SRS) translates the agreed need into detailed system behaviour, interfaces, data requirements, and quality expectations. These documents should be clear enough that a developer can implement the work and a tester can verify it.

A practical BRD may contain the problem statement, project objectives, stakeholders, scope, assumptions, constraints, risks, and acceptance criteria. An SRS may describe functional requirements, non-functional requirements, input and output formats, error handling, external interfaces, and security expectations. For a small classroom project, these can be concise documents rather than lengthy reports.

5.5 High-Level and Low-Level Design

High-level design (HLD) describes the major components of a solution and how they communicate. For a website, it may show the browser interface, application server, database, and external services. Low-level design (LLD) describes the details needed to implement a component, such as field names, validation rules, function responsibilities, or the structure of a page.

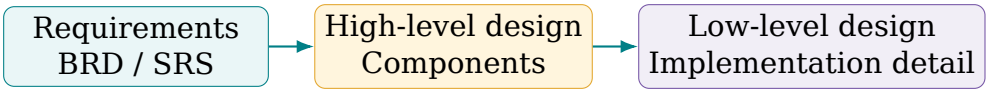


Figure 3.1. Requirements are refined into architecture and implementation detail.

5.6 Design Models and Interfaces

Object-oriented design organises a system around objects that combine data and behaviour. Database design identifies the information that must be stored and the relationships between records. API design defines how separate software components exchange requests and responses. The chosen design should match the project size and should not introduce complexity without a clear need.

Review Questions

1. Distinguish functional requirements from non-functional requirements.
2. Name four stakeholders who may be involved in a website project.
3. Write three acceptance criteria for a school announcements page.
4. Why is it useful to define project scope before coding begins?

Chapter 6

Web Technologies, Browsers and HTTP

6.1 How a Web Page Reaches a Browser

When a visitor enters a website address, the browser uses network services to locate the destination server and request resources. The server returns a response, often containing HTML, CSS, JavaScript, images, fonts, or data. The browser interprets these resources and constructs the visible page. This process may involve many requests rather than a single file.

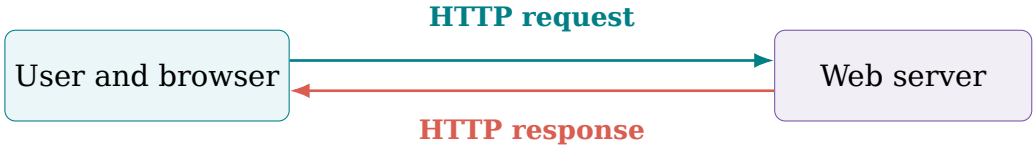


Figure 7.1. A simplified browser-to-server request and response.

6.2 HTTP and HTTPS

HTTP is a protocol used for communication between clients and web servers. A request commonly identifies a method, a resource path, headers, and sometimes a body. A response contains a status code, headers, and usually content. HTTPS protects HTTP communication using TLS, helping protect data from interception or alteration while it travels across the network.

6.2.1 Common Status Codes

- **200 OK:** The request was processed successfully.
- **301 or 302:** The resource has been redirected.
- **400 Bad Request:** The server could not process the request as sent.
- **401 Unauthorized:** Authentication is required or has not succeeded.
- **403 Forbidden:** The server refuses access to the requested resource.
- **404 Not Found:** The requested resource could not be found.
- **500 Internal Server Error:** The server encountered an unexpected condition.

6.3 Browser Developer Tools

Developer tools help inspect HTML elements, view computed CSS, monitor network requests, read console messages, and test responsive layouts. They are useful for learning because a developer can change a style temporarily, observe the result, and then apply a tested correction to the source code.

6.4 Development Tools and Standards

A web developer commonly uses a code editor, browser, developer tools, image editor, version-control system, and a local testing environment. A code editor helps organise files and detect syntax mistakes; a browser displays the result; developer tools help investigate layout, script, and network problems; and version control records changes so that earlier working versions can be recovered.

HTML, CSS, and JavaScript are interpreted by browsers according to web standards. Consistent indentation, meaningful file names, and a clear folder structure make a

project easier to review. A small site might use `index.html`, `styles.css`, `script.js`, and an `images` folder. Keep source files separate by responsibility unless a particular platform requires another structure.

6.5 Common Application Development Models

The Waterfall model organises work into sequential phases such as requirements, design, implementation, testing, and maintenance. It can be straightforward when requirements are stable, but changes discovered late may be expensive. The Iterative model develops a solution through repeated cycles, refining it after feedback. Agile approaches organise work into small increments and encourage frequent collaboration, review, and adaptation.

No single model is best for every project. A small student website may be planned with a simple checklist and improved after each review. A regulated or large system may need formal documentation and approval steps. The useful question is whether the chosen process helps the team deliver the required quality while responding appropriately to change.

Review Questions

1. Describe the basic request-and-response cycle of a web page.
2. Why is HTTPS preferred when information is sent between a browser and server?
3. Explain the difference between a 404 response and a 500 response.
4. Name three tasks that browser developer tools can support.

Chapter 7

Semantic HTML and Accessible Structure

7.1 Meaningful HTML Elements

Semantic HTML uses elements that communicate the role of content. Elements such as `header`, `nav`, `main`, `article`, `section`, and `footer` provide more meaning than using generic containers for every part of a page. Clear structure can improve maintainability and help assistive technologies interpret the document.

```
<header>
  <h1>Community Learning Centre</h1>
  <nav aria-label="Main navigation">
    <a href="#courses">Courses</a>
    <a href="#contact">Contact</a>
  </nav>
</header>
<main>
  <section id="courses">
    <h2>Available courses</h2>
    <p>Explore the current learning options.</p>
  </section>
</main>
<footer>
  <p>Contact the centre for further information.</p>
</footer>
```

7.2 Headings and Content Hierarchy

Headings should describe the topic beneath them and follow a logical hierarchy. A page usually has one main heading, followed by section headings and, where necessary, subheadings. Heading levels should not be selected merely to obtain a desired font size; visual appearance belongs in CSS, while the heading element communicates structure.

7.3 Lists and Grouped Information

HTML supports ordered lists, unordered lists, and description lists. Use an unordered list when the order does not matter, such as a list of required materials. Use an ordered list when the sequence matters, such as installation steps. A description list can pair terms with their explanations.

```
<h2>Preparation checklist</h2>
<ul>
  <li>Open the code editor.</li>
  <li>Create a project folder.</li>
  <li>Save the first HTML page.</li>
</ul>

<h2>Development sequence</h2>
<ol>
  <li>Plan the page.</li>
  <li>Build the structure.</li>
  <li>Test the result.</li>
```

```
</ol>
```

Nested lists are useful when a main item has related sub-items, but deep nesting can make information difficult to scan. Choose the list type according to the meaning of the content, not its default appearance.

7.4 Links, Images and Alternative Text

Link text should tell users what will happen when it is selected. Phrases such as “Read the admission requirements” are more informative than repeated “click here” links. Images that communicate information should have concise alternative text. Decorative images can use empty alternative text so that assistive technologies do not announce unnecessary details.

7.5 Forms and Labels

A form control should have a clear label. Placeholder text is not a replacement for a label because it disappears when the user enters a value and may be difficult to read. Required fields should be indicated clearly, and error messages should explain how to correct a problem.

```
<form>
  <label for="email">Email address</label>
  <input id="email" name="email" type="email" required>
  <button type="submit">Send request</button>
</form>
```

Review Questions

1. What is the purpose of semantic HTML?
2. Explain why heading levels should follow a meaningful order.
3. What makes alternative text useful?
4. Why should a form input have an associated label?

Chapter 8

CSS Selectors, Cascade and Responsive Layout

8.1 Selectors

CSS selectors identify the elements to which a rule applies. Element selectors target a tag name, class selectors target a reusable class, and ID selectors target an element with a specific identifier. Selectors should be chosen to make styles understandable and easy to maintain.

```
/* Element selector */
p { line-height: 1.6; }

/* Reusable class selector */
.notice {
  border-left: 4px solid #007F86;
  padding: 1rem;
}

/* Layout for a group of cards */
.card-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}
```

8.2 Cascade, Specificity and Inheritance

When more than one CSS rule can apply to an element, the cascade determines which declaration wins. Origin, importance, specificity, and source order affect the result. Inherited properties such as text colour may pass from a parent to its children, while properties such as margin do not normally inherit. Overusing highly specific selectors can make future changes difficult.

8.3 Responsive Design

Responsive design helps a layout adapt to different screen sizes. Flexible widths, relative units, CSS Grid, Flexbox, and media queries can be combined to create interfaces that work on phones, tablets, laptops, and larger screens. A responsive page should not merely shrink; content must remain readable and controls must remain usable.

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 1rem;
}
@media (max-width: 700px) {
  .card-grid {
    grid-template-columns: 1fr;
  }
}
```

8.4 Flexbox and Grid

Flexbox is useful for arranging items along one primary axis, such as a navigation row. Grid is useful when a layout needs rows and columns together, such as a card gallery. Choosing the simpler layout model usually makes the code easier to understand.

Review Questions

1. Compare class selectors and ID selectors.
2. What does CSS specificity help determine?
3. Explain one practical difference between Flexbox and Grid.
4. How can a media query improve mobile usability?

Chapter 9

JavaScript Variables, Types and Operators

9.1 Storing Values

Variables give names to values used by a program. JavaScript provides `const` for bindings that should not be reassigned and `let` for bindings that may change. `var` is part of the language but has different scoping behaviour and is usually avoided in new beginner examples.

```
const courseName = "Web Development";
let completedLessons = 3;
completedLessons = completedLessons + 1;

const isReady = completedLessons >= 4;
console.log(courseName, completedLessons, isReady);
```

9.2 Common Data Types

Strings represent text, numbers represent numeric values, booleans represent true or false, and arrays hold ordered collections. Objects group related properties. The value `null` can represent an intentional absence of a value, while `undefined` commonly indicates that a value has not been assigned or is missing.

9.3 Operators and Comparisons

Arithmetic operators perform calculations. Comparison operators produce Boolean results, and logical operators combine conditions. Strict equality (===) compares values without automatically converting different types, which makes it a good default for most comparisons.

9.4 Readable and Safe Code

Descriptive variable names, consistent indentation, and small expressions make code easier to review. Validate user-provided data before relying on it. Never treat browser-side validation as the only protection for a system that stores sensitive data; server-side checks are also required when a server is involved.

Review Questions

1. Compare `let` and `const`.
2. Name four JavaScript data types and give an example of each.
3. Why is strict equality often preferred?
4. Why should important validation not rely only on browser-side code?

Chapter 10

Conditions, Loops, Functions and Arrays

10.1 Conditional Decisions

Conditional statements allow a program to choose an action based on a condition. An `if/else` statement can validate input, choose a message, or control which content is shown. Conditions should be written clearly and tested for both expected and unexpected values.

```
const score = 76;
if (score >= 80) {
  console.log("Excellent");
} else if (score >= 50) {
  console.log("Pass");
} else {
  console.log("Needs more practice");
}
```

10.2 Loops

Loops repeat an operation while a condition is true or for each item in a collection. A loop needs a clear stopping condition; otherwise it may continue indefinitely. For arrays, methods such as `forEach`, `map`, and `filter` can express common operations when used appropriately.

10.3 Functions

A function groups instructions into a reusable unit. Parameters receive input, and a return value provides a result. Small functions are easier to test than one long block of mixed responsibilities.

```
function calculateAverage(values) {
  if (values.length === 0) return 0;
  const total = values.reduce((sum, value) => sum + value, 0);
  return total / values.length;
}
console.log(calculateAverage([70, 80, 90]));
```

10.4 Arrays and Objects

Arrays are suitable for ordered lists such as course names. Objects are suitable for related attributes such as a course title, duration, and level. Choosing an appropriate data structure reduces repetition and helps the program represent information clearly.

10.5 Working with Strings

A string is a sequence of text characters. JavaScript provides properties and methods for checking length, changing letter case, finding text, extracting a portion, and joining values. String methods usually return a result rather than changing the original string, so store the returned value when a change is needed.

```
const learner = "  Siva Sai  ";
```

```
const cleaned = learner.trim();
const greeting = "Welcome, " + cleaned + "!";
console.log(greeting);
console.log(cleaned.toUpperCase());
```

When processing names or messages, consider empty input and extra spaces. Do not assume that every text value is valid simply because it is a string; validation should match the purpose of the field.

Review Questions

1. Explain how an `if`/`else` statement makes a decision.
2. What is the purpose of a loop's stopping condition?
3. Define function parameters and return values.
4. Give one example of information that is better represented by an object than by separate variables.

Chapter 11

DOM, Events and Form Interaction

11.1 The Document Object Model

The Document Object Model (DOM) represents a page as a tree of objects. JavaScript can select elements, read or update text, change classes, and respond to events. Changes should be made deliberately so that the page remains understandable and accessible.

```
const button = document.querySelector("#show-message");
const message = document.querySelector("#message");

button.addEventListener("click", () => {
  message.textContent = "Your action was received.";
});
```

11.2 Events

Events describe interactions such as clicks, keyboard input, form submission, and changes to a field. An event listener connects an event to a function. For forms, listening to the submit event makes it possible to validate data before displaying a result or sending a request.

11.3 Validation and Feedback

Good validation explains what needs correction. It should preserve user input where possible, identify the field with an issue, and avoid relying on colour alone to communicate errors. A successful action should also provide clear confirmation.

11.4 Safe DOM Updates

When displaying untrusted text, prefer `textContent` rather than inserting arbitrary HTML with `innerHTML`. This reduces the risk of interpreting user input as markup or script. More complex applications need additional security controls appropriate to their data and architecture.

11.5 Drawing with the HTML Canvas

The HTML `canvas` element provides a drawing surface that JavaScript can control. It is useful for simple charts, sketches, or interactive visual demonstrations. Canvas content is drawn with script and is not automatically exposed as ordinary text to assistive technology; provide a text explanation or an accessible alternative where the drawing conveys important information.

```
<canvas id="demo-canvas" width="320" height="140">
  A simple coloured rectangle illustration.
</canvas>
```

```
const canvas = document.querySelector("#demo-canvas");
const context = canvas.getContext("2d");

context.fillStyle = "#007F86";
context.fillRect(20, 20, 180, 70);
```

```
context.fillStyle = "#17365D";
context.font = "16px sans-serif";
context.fillText("Web Developer", 30, 115);
```

The canvas dimensions define its drawing surface. CSS can control how large it appears on the page, but changing only its displayed size may scale the drawing. For beginner projects, keep the example small and test it in the browser.

11.6 Using JavaScript with Images

JavaScript can respond to a button click by changing an image's source, updating alternative text, or adjusting presentation classes. Alternative text should continue to describe the image's purpose after a change. Images should have appropriate dimensions and should not be stretched in ways that distort their content.

```
<img id="course-image"
     src="images/html-basics.jpg"
     alt="HTML code displayed in an editor">
<button id="change-image" type="button">
  Show CSS image
</button>
```

```
const image = document.querySelector("#course-image");
const changeButton = document.querySelector("#change-image");

changeButton.addEventListener("click", () => {
  image.src = "images/css-basics.jpg";
  image.alt = "CSS rules displayed in an editor";
});
```

The image paths must point to files that actually exist in the project. If an image cannot load, provide a useful fallback or ensure that the page still communicates its essential information.

Review Questions

1. What does the DOM represent?
2. Describe how an event listener works.
3. Give two characteristics of helpful form validation.
4. Why is `textContent` a safer default for displaying plain user input?

Chapter 12

Testing, Debugging and Quality Assurance



Figure 12.1. Careful development and review are part of building reliable websites. Photo: Free-Photos / Pixabay, CC0 1.0.

12.1 Why Testing Matters

Testing checks whether a website behaves as expected and helps detect defects before release. Testing should cover more than whether a page appears on one developer's computer. Different screen sizes, keyboard use, input errors, navigation paths, and browser behaviours can reveal problems that are otherwise missed.

12.1.1 Useful Test Categories

- **Functional testing:** Confirms links, forms, buttons, and core features behave correctly.
- **Visual testing:** Checks alignment, spacing, contrast, and layout consistency.
- **Responsive testing:** Checks content at narrow and wide viewport sizes.
- **Accessibility testing:** Checks keyboard navigation, labels, headings, and alternative text.
- **Compatibility testing:** Checks behaviour in supported browsers and devices.
- **Regression testing:** Rechecks existing features after a change.

12.2 A Practical Debugging Method

First reproduce the issue and record the steps. Next inspect the relevant element, console message, network response, or source code. Form a small hypothesis, change one thing at a time, and repeat the test. Once fixed, check nearby features to make sure the correction did not introduce another defect.

12.3 Test Case Example

Field	Example
Test objective	Verify required email input.
Starting condition	Contact form is open.
Action	Submit without entering an email address.
Expected result	The form identifies the missing required field and does not report a successful submission.
Observed result	Record what actually happened.
Status	Pass or fail after comparison with the expected result.

Review Questions

1. Why is responsive testing important?
2. Distinguish functional testing from regression testing.
3. List the steps in a systematic debugging process.
4. What information should a useful test case record?

Chapter 13

Accessibility, Performance and Secure Publishing

13.1 Accessibility Essentials

Accessible design helps people with different abilities use a website. Provide clear headings, descriptive links, form labels, sufficient colour contrast, keyboard access, and useful alternative text. Interactive controls should show a visible focus indicator. Automated checks can help, but manual keyboard and content checks remain valuable.

13.2 Performance Basics

Large images, unnecessary scripts, repeated network requests, and complex layouts can slow a page. Resize images appropriately, use modern image formats where supported, avoid loading resources that are not needed, and keep code organised. Performance should be evaluated with measurements rather than guesses.

13.3 Security Awareness

Use HTTPS for published sites. Do not place passwords, private API keys, or confidential tokens in public HTML, CSS, JavaScript, or source repositories. Client-side code is visible to visitors. If a feature requires a secret credential, it generally belongs in a protected server-side environment. Keep software dependencies maintained and collect only the information a site actually needs.

13.4 Publishing Checklist

Before publishing, verify that links work, page titles are meaningful, images have suitable alternative text, forms give clear feedback, layouts work on mobile screens, and no test credentials or private data remain in the repository. Confirm the correct files are deployed and keep a copy of the source so that later updates can be made safely.

Review Questions

1. List five practices that improve accessibility.
2. Name three common causes of slow web pages.
3. Why should secret API keys not be stored in browser-side JavaScript?
4. Create a short checklist for publishing a small informational website.

Chapter 14

Integrated Mini Project: Learning Centre Website

14.1 Project Brief

Create a small informational website for a learning centre. The website should display the centre name, a list of courses, basic contact information, and a contact form interface. The project demonstrates the combined roles of HTML for structure, CSS for presentation, and JavaScript for interaction.

14.2 Suggested Page Structure

- Header containing the site name and primary navigation.
- Main section describing the centre and its courses.
- Course cards with titles, brief descriptions, and duration.
- Contact section with labelled form fields.
- Footer containing non-sensitive contact and copyright information.

14.3 Implementation Plan

Begin by creating semantic HTML. Add CSS rules for typography, colour, spacing, and a responsive card layout. Add JavaScript only where it improves the user experience, such as providing a clear message after a valid client-side form submission. If the project is later connected to a real service, server-side validation and secure handling of submitted information will be required.

14.4 Evaluation Rubric

Criterion	Marks
Semantic structure and meaningful headings	20
Readable and consistent visual design	20
Responsive layout	20
Working interaction and useful feedback	15
Accessibility checks	15
Code organisation and testing notes	10
Total	100

Review Questions

1. Identify the HTML, CSS, and JavaScript responsibilities in the project.
2. Describe how you would test the site on a narrow mobile screen.
3. What security changes would be required before collecting real contact details?
4. Suggest two improvements that could be included in a future version.